

Algorithms for Strings and Sequences: Pairwise Alignment

Stefano Beretta, University of Milan – Bicocca, Milan, Italy

© 2018 Elsevier Inc. All rights reserved.

Introduction

The problem of aligning sequences is probably one of the classical and most important tasks in bioinformatics. In fact, in many studies one needs to compare strings or search for motifs such as, for example, nucleotide reads obtained from a sequencing process that must be aligned against a reference genome, or sequences of amino acids that must be compared to each other.

Recent advances in sequencing technologies have lead to a revolution in bioinformatics, since, with the introduction of Next-Generation Sequencing methods, a huge amount of data can be produced at a cheaper cost (Schuster, 2008). Thanks to this, it is now possible to perform new studies that were not even thinkable up to some years ago, such as the identification of organisms from a metagenomic survey (Rodrigue *et al.*, 2010; America, 2009), or the identification of structural variations present in genomes (Medvedev *et al.*, 2009; Tattini *et al.*, 2015). Although the computational problems arising from these studies require new algorithms and methods that are able to efficiently compute solutions by taking advantage of the different nature of the data, pairwise sequence alignment is still widely performed. In fact, after assembling transcripts or new genomes (e.g., from new species or from new individuals), it is important to determine the degree of similarity between two sequences, since high sequence similarity usually implies that they could have the same function or properties. For example, pairwise sequence alignment could also help in identifying the function of an unknown gene, by finding a similar gene (in terms of sequence composition) of known function.

In this work we will focus on the pairwise sequence alignment problem, in which two biological sequences are compared in order to identify their similarity. Intuitively, the process of aligning two sequences consists of lining them up, by possibly inserting spaces or gaps, while keeping the original order of the symbols, so that each symbol of one sequence is mapped to a symbol of the other sequence or to a null character or gap. More precisely, after introducing some basic notions related to this problem, we will present two types of pairwise alignments: global and local. The former alignment aims at finding the best alignment over the full length of the two input sequences, while the latter is used to identify the most similar parts (substrings) of the two sequences. We will also present two approaches, based on dynamic programming, to solve these two problems, and we will describe scoring functions used to achieve specific results in the alignment of biological sequences.

Background

In this section, we introduce the basic concepts of strings and pairwise alignment.

Let Σ be a non-empty finite set of symbols. A typical choice when dealing with nucleotide sequences is $\Sigma = \{A, C, G, T\}$. A string, $s = s_1 \dots s_n$, over an alphabet, Σ , is a sequence of symbols of Σ . We would like to point out that, although in some situations these terms are used as synonyms, in the rest of the paper we will use the term “sequence” only in the biological context, while for the mathematical and algorithmic ones we will use the term “string”.

Given a string, s , over Σ , we denote by $|s|$ the length of s , and by s_i , with $1 \leq i \leq |s|$, the symbol of s at position i . Moreover, we denote by $s_{i,j}$, with $1 \leq i < j \leq |s|$, the substring of s starting at position i and ending at position j , that is, the string $s_i \dots s_j$. We also denote by ϵ the empty string.

Consider now a symbol ‘-’, not in Σ , used to denote *gaps*, that is, ‘-’ $\notin \Sigma$. In order to define an alignment between two strings, s and t , over Σ , we consider the alphabet $\Sigma' = \Sigma \cup \{-\}$. At this point we are able to define the concept of alignment. Given two strings, s and t , over Σ , the (global) *alignment* of s and t is a pair (s', t') in which (i) s' is obtained from s by possibly inserting gaps, (ii) t' is obtained from t by possibly inserting gaps, (iii) $|s'| = |t'|$, and (iv) there is no position i , with $1 \leq i \leq |s'|$, such that both s'_i and t'_i are gaps.

As an example, consider the two strings, $s = \text{ATACAC}$ and $t = \text{AAGCGC}$, over the alphabet $\Sigma = \{A, C, G, T\}$; a possible alignment of s and t is:

$$s' = A \quad T \quad A \quad - \quad C \quad A \quad C$$
$$t' = A \quad - \quad A \quad G \quad C \quad G \quad C$$

In this example we obtained s' from s by adding a gap between the first A and C symbols (4th position), and we obtained t' from t by adding a gap between the two A symbols (2nd position). Considering this representation, in each position $i \in \{1, \dots, |s'| = |t'|\}$ of the alignment, we can find one of the following four possible situations:

- *insertion*: $s'_i = \text{'-'}'$, corresponding to the fact that a gap has been inserted in the first string (position 4 of the example alignment);
- *deletion*: $t'_i = \text{'-'}'$, corresponding to the fact that a gap has been inserted in the second string (position 2 of the example alignment);

- *match*: $s_i = t_i$, corresponding to the fact that the symbol in that position is the same in both the strings (positions 1, 3, 5, and 7 of the example alignment);
- *mismatch*: $s_i \neq t_i$, corresponding to the fact that the symbols in that position are different (position 6 of the example alignment).

It must be noticed that, from this definition, two input strings can always be aligned. Anyway, since the goal of pairwise alignment is to assess the similarity of the two input strings, it has to be scored. A common way to do this is to assign a value to each possible situation that can happen in a position when aligning two symbols, that is, to assign score values to insertion and deletion (gap alignments), match, and mismatch. Then, the score of the alignment is computed by summing up the values of the score at each position of the alignment. More precisely, we denote by w the *scoring function* that assigns to each possible combination of symbols in Σ' a score, that is $w : \Sigma' \times \Sigma' \rightarrow \mathbb{N}$ (or $\mathbb{Q}$, if we allow rational values).

From this point of view, it is possible to measure the *distance* or the *similarity* between the two input strings, depending on the specific values assigned by the scoring function. An example of the former type of measure is the *edit distance* in which the distance between the two strings is determined by the minimum number of edit operations required to transform the first string into the second one. On the other hand, different similarity measures can be adopted to quantify how much the two given input strings are similar. Recalling the previous example, it is possible to define a *similarity scoring function* by assigning a value 1 to match positions, and value -1 to mismatch and gap alignments, that is, to both insertions and deletions. The overall score of the alignment in the example is 1 (4 - 3). More precisely, in this case the (similarity) scoring function w is:

$$w(s'_i, t'_j) = \begin{cases} 1 & \text{if } s'_i = t'_j \wedge s'_i \neq '-' \\ -1 & \text{otherwise} \end{cases}$$

with $s'_i, t'_j \in \Sigma'$, and $1 \leq i, j \leq |s'| = |t'|$. In general, with this or any other similarity scoring function, higher values of the score will correspond to "better" alignments, i.e., shorter edit distances, and the optimal global pairwise alignment will have the highest score.

As anticipated, another possible choice for the scores of the positions is the *edit distance*, which measures the number of insertions, deletions, and substitutions required to transform one string into the other. It is interesting to notice that, when dealing with biological sequences, each edit operation corresponds to a biological operation: insertion, deletion, and mutation, respectively. A specific type of edit distance is the *Levenshtein distance*, which assigns 0 to matches, and 1 to mismatches and gap positions. In this case the scoring function modeling the edit distance w_{ed} is:

$$w_{ed}(s'_i, t'_j) = \begin{cases} 0 & \text{if } s'_i = t'_j \wedge s'_i \neq '-' \\ 1 & \text{otherwise} \end{cases}$$

with $s'_i, t'_j \in \Sigma'$, and $1 \leq i, j \leq |s'| = |t'|$. Since this is a distance measure, it corresponds to counting the number of non-matching positions, which means that the optimal alignment is the one that minimizes the score.

Obviously, there may be more than one solution, i.e., alignment, with the same optimal score.

In Section Methodologies a more detailed explanation of the *scoring function*, which assigns a score value to each possible combinations of symbols of the alphabet Σ' used in the alignment, will be provided.

Methodologies

As anticipated in Section Background, the goal of pairwise alignment of two input strings is to find the alignment having the best score. In this context, we can distinguish two type of alignments, namely *global* and *local*. In particular, the former type aligns each position of the two input strings by computing the best score that includes every character in both sequences, while the latter type aims at finding the regions of the input strings, that is, substrings, with the best match. To solve these problems, there exist different techniques, ranging from manual alignments made by inspection of the strings, to comparison using the dotplot matrix. Anyway, the most widely used techniques for the pairwise alignment, both global and local, rely on dynamic programming algorithms, in which the optimal alignment is obtained recursively by using the previously computed solutions of smaller substrings.

In the rest of this section, we will describe the *Needleman-Wunsch* algorithm (Needleman and Wunsch, 1970) which solves the global pairwise alignment problem, and the *Smith-Waterman* algorithm (Smith and Waterman, 1981), which solves the local pairwise alignment computation. Both algorithms are based on dynamic programming (Bellman, 1954). Moreover, we will also discuss different scoring functions which are commonly adopted to evaluate the alignments of biological sequences, i.e., nucleotide or protein.

Before presenting the two aforementioned algorithms, we will briefly explain the main concepts of dynamic programming, which is employed by both the methods. The idea is to recursively decompose the problem into subproblems and compute the solutions of the subproblems starting from partial solutions of smaller subproblems, computed at earlier stages. The dynamic programming technique can be applied when a solution to a problem can be obtained as a series of consecutive steps, each one working on a continuously growing subproblem. In this way, starting from the smallest subproblems, which are directly solved, subproblems are solved by combining previously computed solutions, following back the recursive decomposition. The solution of the final stage contains the overall solution.

The idea of applying this technique to the pairwise alignment problem of two input strings, s and t , is to consider the subproblems of aligning substrings, $s_{1,i}$ and $t_{1,j}$, with increasing values of i , ranging from 1 to $|s|$, and j , ranging from 1 to $|t|$. At each step, a new pair of positions, i and j , is considered, and the score $w(s_i, t_j)$ of their alignment is added to the score of the best alignment computed in the previous step. In this way, all the best alignments of two substrings ending at the last considered position can be stored in a matrix, as well as their scores. One of the key points about this dynamic programming method is that it is guaranteed to find at least one optimal alignment of the strings, s and t , for a given scoring function. On the other hand, although this technique can be used to simultaneously align several strings, it becomes quite slow, and is therefore not practical for large numbers of strings.

In the next two sections, we describe in detail the algorithmic procedures for the computation of the optimal global and local pairwise alignments. In this presentation, we adopt the scoring function w introduced in Section Background. More precisely, we define the following scoring function:

$$w(s'_i, t'_j) = \begin{cases} 1 & \text{if } s'_i = t'_j \wedge s'_i \neq '-' & (\text{Match}) \\ -1 & \text{if } s'_i \neq t'_j \wedge s'_i \neq '-' \wedge t'_j \neq '-' & (\text{Mismatch}) \\ -1 & \text{if } s'_i = '-' \vee t'_j = '-' & (\text{Gap}) \end{cases}$$

with $s'_i, t'_j \in \Sigma'$, and $1 \leq i, j \leq |s'| = |t'|$. With this similarity scoring function, the optimal alignment has the highest score and, consequently, both global and local algorithmic procedures maximize it.

Global Alignment

In this section we will describe the dynamic programming algorithm proposed by Needleman and Wunsch to solve the problem of computing an optimal global alignment between two strings (Needleman and Wunsch, 1970). Anyway, although we refer to the algorithm described in the rest of this section as that of Needleman and Wunsch, this is a bit imprecise. In fact, the first dynamic programming algorithm to solve the global pairwise alignment was introduced by Needleman and Wunsch in 1970 (having complexity $O(n^3)$ in time, although an efficient implementation can achieve an $O(n^2)$ in time), but the one described here is the one proposed later by Sellers (1974). This algorithm, which is also based on dynamic programming, requires $O(n^2)$ time, and uses a linear model for gap penalties (see later).

Let s and t be two strings over the alphabet, Σ , such that $|s| = n$ and $|t| = m$. The idea of the algorithm is to compute the optimal alignments of the prefixes of s and t , and to store the score in a matrix, M , having size $(n + 1) \times (m + 1)$, in which the '+' 1' is used to represent the initial empty substring ϵ . More precisely, $M[i, j]$ represents the score of an optimal alignment of two substrings $s_{1,i} = s_{\{1\} \dots \{i\}}$ and $t_{1,j} = t_{\{1\} \dots \{j\}}$. The dynamic programming algorithm computes $M[i, j]$ for i ranging from 0 (empty prefix) to n , and j ranging from 0 to m . At the end of the procedure, $M[n, m]$ is the score of the optimal global alignment of s and t .

Now, let us define the initialization of the dynamic programming, that is, the score of aligning the empty prefixes of the strings. More precisely, we start by setting $M[0, 0] = 0$, corresponding to the score of aligning two empty prefixes ϵ . Then, since we adopt a length-dependent model for gap penalties (Sellers, 1974), for every $i \in \{1, \dots, n\}$, set $M[i, 0] = i \cdot w(s_i, '-')$, corresponding to the score of aligning a prefix of s having length i , that is $s_{1,i}$, with the empty string ϵ . The same applies also for $j \in \{1, \dots, m\}$ for which we set $M[0, j] = j \cdot w('-', t_j)$, corresponding to the score of aligning a prefix of t having length j , that is $t_{1,j}$, with the empty string ϵ .

Now, it is possible to describe the recurrence. The idea, when considering the cell $M[i, j]$ of the matrix, which corresponds to the score of the optimal alignment of the prefixes $s_{1,i}$ and $t_{1,j}$, is that we take advantage of the previously computed scores for $i-1$ and $j-1$. Since, by definition, the goal is to maximize the final alignment score, we select the maximum score value among that of the alignments of the three possible prefixes of $s_{1,i}$ and $t_{1,j}$. The score of the alignment of the prefixes $s_{1,i}$ and $t_{1,j}$ is computed as the maximum of the following three cases:

- (1) s_i and t_j are aligned, and the score is that of the alignment of prefixes $s_{1,i-1}$ and $t_{1,j-1}$ plus the score $w(s_i, t_j)$ which is positive for a match and negative for a mismatch;
- (2) there is a deletion in t ; s_i is therefore aligned with a gap $'-'$. The score is that of the alignment of prefixes $s_{1,i-1}$ and $t_{1,j}$ plus the score of the gap alignment $w(s_i, '-')$;
- (3) there is an insertion in t ; t_j is aligned with a gap $'-'$. The score is that of the alignment of prefixes $s_{1,i}$ and $t_{1,j-1}$ plus the score of the gap alignment $w('-', t_j)$.

These cases are formalized in the following equation:

$$M[i, j] = \max \begin{cases} M[i-1, j-1] + w(s_i, t_j) & (1) \\ M[i-1, j] + w(s_i, '-') & (2) \\ M[i, j-1] + w('-', t_j) & (3) \end{cases}$$

for $i \in \{2, \dots, n\}$ and $j \in \{2, \dots, m\}$. Fig. 1 offers a graphical representation of the general case of the dynamic programming recurrence to compute the score $M[i, j]$ starting from the previously computed scores.

Finally, the score $Score_{OPT}$ of the optimal alignment of the two input strings s and t is in the cell $M[n, m]$:

$$Score_{OPT} = M[n, m]$$

	0	1	...	$j-1$	j	...	m
0							
1							
$\vdots$							
$i-1$				$M[i-1, j-1]$	$M[i-1, j]$		
i				$M[i, j-1]$	$M[i, j]$		
$\vdots$							
n							

Fig. 1 Graphical representation of the computation of $M[i, j]$ in the matrix M using a linear gap penalties model (Sellers, 1974). Here, the 3 previously computed values on which the recurrence is based are highlighted, namely $M[i-1, j-1]$, $M[i-1, j]$, and $M[i, j-1]$. Labels on the arrows report the additional score that is added, corresponding to the pair of aligned symbols.

$s \backslash t$	ϵ	A	A	G	C	G	C
ϵ	0	-1	-2	-3	-4	-5	-6
A	-1	1	0	-1	-2	-3	-4
T	-2	0	0	-1	-2	-3	-4
A	-3	-1	1	0	-1	-2	-3
C	-4	-2	0	0	1	0	-1
A	-5	-3	-1	-1	0	0	-1
C	-6	-4	-2	-2	0	-1	1

$s \backslash t$	ϵ	A	A	G	C	G	C
ϵ	·	←	←	←	←	←	←
A	↑	↖	↖	←	←	←	←
T	↑	↑	↖	↖	↖	↖	↖
A	↑	↖	↖	←	←	←	←
C	↑	↑	↑	↖	↖	←	↖
A	↑	↖	↖	↖	↑	↖	↖
C	↑	↑	↑	↖	↖	↖	↖

Fig. 2 Example of the computation of optimal alignment matrices for the two input strings $s=ATACAC$ and $t=AAGCGC$. The left matrix represents the score of the optimal (maximum) alignment, while the right matrix contains the arrows to keep track of the best choice done to obtain the optimal score. The grey path corresponds to the optimal final alignment. In both the matrices, instead of reporting the values of $i \in \{0, \dots, 6\}$ and $j \in \{0, \dots, 6\}$ as row and column names, respectively, we used the corresponding symbols of the strings, $\{\epsilon, s_1, \dots, s_6\}$ and $\{\epsilon, t_1, \dots, t_6\}$, respectively.

Anyway, in order to reconstruct the optimal alignment (or alignments if there is more than one), it is necessary to keep track, for each pair of symbols at positions i and j , from which of the three aforementioned cases was obtained the value $M[i, j]$ during the dynamic programming. More precisely, since to compute $M[i, j]$ we select the maximum value among the 3 possible cases of the dynamic programming recurrence, in order to reconstruct the optimal alignment we have to compute from which value we obtained the maximum score $M[i, j]$ among $M[i-1, j-1]$, $M[i-1, j]$, and $M[i, j-1]$ (see Fig. 1). A possible way to represent this choice is to use arrows pointing to the corresponding matrix cell so that, once the matrix is completed, a path can be traced from the cell with the optimal value $Score_{OPT}$ (i.e., $M[n, m]$) to $M[0, 0]$, and the alignment reconstructed from the pairs of letters aligned at each step. This reconstruction process is usually called *traceback*. In fact, there is a pair of symbols corresponding to each possible arrow at a given position i, j of the matrix:

- ↖ corresponds to the alignment of symbols s_i and t_j ;
- ↑ corresponds to the alignment of symbols s_i and $'-'$;
- ← corresponds to the alignment of symbols $'-'$ and t_j .

The two matrices in Fig. 2 show the scores and the corresponding arrows for the computation of the optimal alignment of the two input strings used as an example in Section Background: $s=ATACAC$ and $t=AAGCGC$. By slightly abusing the notation, the row/column indices can be used as the symbols of the input string s/t , instead of the positions $i \in \{0, \dots, 6\}$ and $j \in \{0, \dots, 6\}$. Following the path in the right matrix of Fig. 2 we can reconstruct an optimal alignment having score equal to 1:

$$s' = A \quad T \quad A \quad - \quad C \quad A \quad C$$

$$t' = A \quad - \quad A \quad G \quad C \quad G \quad C$$

We would like to point out that, for ease of exposition, in the example in Fig. 2 only a single arrow for each cell of the matrix is shown, whereas, in reality, an equal optimal score can frequently be obtained from multiple “directions”. This is due to the fact

that there could be multiple alignments of the two input strings having the same optimal score, $Score_{OPT}$. For this reason, if it is necessary to reconstruct all the optimal alignments of two given strings, for each cell, all the possible optimal choices should be stored. Moreover, the traceback step used to reconstruct the optimal alignment should explore all the possible paths (not just a single one) from $M[n, m]$ to $M[0, 0]$.

Local Alignment

In this section, we will describe the algorithm proposed by Smith and Waterman, which is used to solve the problem of computing the optimal local alignment of two input strings (Smith and Waterman, 1981). The goal of this procedure is to find the substrings of the input strings that are the most similar, with respect to a given scoring function, which usually represents a similarity measure (like the one used in Section Global Alignment to describe the global alignment).

The Smith–Waterman algorithm is based on a dynamic programming recurrence similar to that used to find the global pairwise alignment, and follows the same idea: compute a matrix with the optimal scores of the alignments of the prefixes of the input strings, keeping track of the previous optimal partial alignments, and at the end follow a traceback path to reconstruct an optimal alignment.

Formally, given two input strings, s and t , having lengths n and m , respectively, we compute a $(n + 1) \times (m + 1)$ matrix M , in which $M[i, j]$, with $i \in \{0, \dots, n\}$ and $j \in \{0, \dots, m\}$, represents the optimal (maximum) score of a local alignment between the substrings $s_{1,i}$ and $t_{1,j}$. Here, the interpretation of the matrix values is slightly different with respect to that of the global pairwise alignment: $M[i, j]$ represents the score of the best alignment of a suffix of $s_{1,i}$ and a suffix of $t_{1,j}$. In the local pairwise alignment matrix, the first row and the first column, i.e., those with index 0, are set to 0, since there is no penalty in introducing gaps. Moreover, as anticipated, for each cell of the matrix we store the path taken to obtain the maximum score value (i.e., through an arrow to the corresponding cell) in order to reconstruct the best local alignment.

The recurrence starts by setting the first row and the first column of the matrix M with 0, that is, $M[0, j] = 0$, with $j \in \{0, \dots, m\}$, and $M[i, 0] = 0$, with $i \in \{0, \dots, n\}$. Now, the recursive case of the recurrence, is:

$$M[i, j] = \max \begin{cases} M[i - 1, j - 1] + w(s_i, t_j) \\ M[i - 1, j] + w(s_i, -) \\ M[i, j - 1] + w(-, t_j) \\ 0 \end{cases}$$

for $i \in \{1, \dots, n\}$ and $j \in \{1, \dots, m\}$.

Notice that, while an alignment can be produced for any scoring system, if the average score for comparing the letters is not below zero or, more in general, for all positive scoring systems, it will result in a global alignment (and not in a local one).

and for an all positive scoring system

The value 0 at position (i, j) , i.e., $M[i, j] = 0$, indicates that a new alignment starts at this position. In fact, if the score of the best alignment up to (i, j) is negative, then it is better to start a new one. Differently from the global pairwise alignment procedure described before, all the negative scores are replaced by 0 in the matrix M .

Finally, the score of the optimal local pairwise alignment between s and t can be found by searching for the highest value in the matrix M :

$$Score_{OPT} = \max\{M[i, j] : 1 \leq i \leq n, 1 \leq j \leq m\}$$

After that, similarly to the global alignment procedure, in order to reconstruct a local alignment having optimal score, it is necessary to follow the traceback path, from the matrix cell of M having score $Score_{OPT}$ up to a cell having score equal to 0.

The matrices in Fig. 3 show an example of optimal local alignment for the two input strings $s = ACTAAG$ and $t = CTCAAT$ w.r.t. the scoring function introduced at the beginning of Section Methodologies. More precisely, the left matrix reports the score values obtained with the dynamic programming recurrence, while the matrix on the right shows the arrows corresponding to the choices made, that is, the cells from which the values have been obtained. In this latter matrix, we use a dot (i.e., $\cdot$) for those cells with a 0 score obtained through the last case of the dynamic programming recurrence.

On the other hand, as in the example of the global pairwise alignment shown in Section Global Alignment, only one arrow for each cell is shown, following the order shown in the definition of the recurrence, when there is more than one possible choice. Finally, the grey path corresponds to the optimal local alignment having $Score_{OPT} = 3$:

$$\begin{aligned} s' &= C & T & - & A & A \\ t' &= C & T & C & A & A \end{aligned}$$

Extension to Affine Gap Model

Now, let us analyze the introduction of gap symbols in the alignment. Both the algorithms described in Section Global Alignment and Section Local Alignment use a linear gap model to assign a score to the alignment of a symbol in the alphabet Σ with the gap '-'. This usually consists of a negative score associated with a gap being dependent on the number of successive gap characters.

$s \backslash t$	ϵ	C	T	C	A	A	T
ϵ	0	0	0	0	0	0	0
A	0	0	0	0	1	1	0
C	0	1	0	1	0	0	0
T	0	0	2	1	0	0	1
A	0	0	1	1	2	1	0
A	0	0	0	0	2	3	2
G	0	0	0	0	1	2	2

$s \backslash t$	ϵ	C	T	C	A	A	T
ϵ	.	.	.	.	.	.	.
A	.	.	.	.	$\nwarrow$	$\nwarrow$	$\leftarrow$
C	.	$\nwarrow$	$\leftarrow$	$\nwarrow$	$\uparrow$	$\uparrow$	$\nwarrow$
T	.	$\uparrow$	$\nwarrow$	$\leftarrow$	$\nwarrow$	.	$\nwarrow$
A	.	.	$\uparrow$	$\nwarrow$	$\nwarrow$	$\nwarrow$	$\uparrow$
A	.	.	$\uparrow$	$\uparrow$	$\nwarrow$	$\nwarrow$	$\leftarrow$
G	.	.	.	.	$\uparrow$	$\uparrow$	$\nwarrow$

Fig. 3 Example of the computation of optimal local alignment matrices for the two input strings $s = ACTAAG$ and $t = CTCAAT$. The left matrix represents the score of the optimal (maximum) alignment, while the right matrix contains the arrows to keep track of the best choice done to obtain the optimal score. The grey path corresponds to the optimal local final alignment. In both the matrices, instead of reporting the values of $i \in \{0, \dots, 6\}$ and $j \in \{0, \dots, 6\}$ as row and column names, respectively, we used the corresponding symbols of the strings, $\{\epsilon, s_1, \dots, s_6\}$ and $\{\epsilon, t_1, \dots, t_6\}$, respectively.

More precisely, in this *length-dependent* gap model, the score of introducing a sequence of k consecutive gap symbols in an aligned string s is $k \cdot w(t_j, '-')$, with $1 \leq j \leq m$, that is k times the score of aligning a symbol t_j of t with a gap (Sellers, 1974).

Anyway, in many biological analyses (but also in other research fields) having k consecutive gaps is preferable to having k gaps scattered along the strings in different positions. For this reason, a different gap model, which distinguishes between the score for opening the gap (length independent score) and the cost proportional to the gap length, was introduced by Gotoh (1982). This model, also known as the *affine* gap cost model, penalizes alignments with many scattered gap positions more than alignments with more consecutive ones.

Now, given a gap opening cost h , and a gap proportional cost similar to that used in Section Global Alignment and Section Local Alignment, the dynamic programming recursion to compute the global pairwise alignment can be modified as follows:

$$\begin{aligned}
 M[i, j] &= \max \begin{cases} M[i-1, j-1] + w(s_i, t_j) \\ I_s[i-1, j-1] + w(s_i, t_j) \\ I_t[i-1, j-1] + w(s_i, t_j) \end{cases} \\
 I_s[i, j] &= \max \begin{cases} M[i-1, j] + h + w(s_i, '-') \\ I_s[i-1, j-1] + w(s_i, '-') \end{cases} \\
 I_t[i, j] &= \max \begin{cases} M[i, j-1] + h + w('-', t_j) \\ I_t[i, j-1] + w('-', t_j) \end{cases}
 \end{aligned}$$

for $i \in \{1, \dots, n\}$ and $j \in \{1, \dots, m\}$. The matrices are initialized by setting:

- $M[0, 0] = 0$;
- $I_s[i, 0] = h + i \cdot w(s_i, '-')$, with $i \in \{1, \dots, n\}$;
- $I_t[0, j] = h + j \cdot w('-', t_j)$, with $j \in \{1, \dots, m\}$;
- all the other cells in the first rows and first columns of matrices M , I_s , and I_t to $-\infty$.

The additional matrices I_s and I_t are used to deal with the introduction of gaps in the alignment, to distinguish between the case in which a new gap is opened and the case in which it is extended.

Notice that the Smith-Waterman dynamic programming recurrence can be modified in the same way to adopt the affine gap model. Anyway, in both cases, the time complexity of the two algorithms still remains $O(n^2)$.

Scoring Functions

In this section we briefly discuss different scoring functions that are used in real case studies to perform pairwise alignments, especially when dealing with nucleotide and protein sequences. More precisely, another way to represent a scoring function $w : \Sigma' \times \Sigma' \rightarrow \mathbb{N}$ which assigns a value to each possible alignment of two symbols, is through a *scoring matrix*, that is a matrix having size $|\Sigma'| \times |\Sigma'|$. In this way, to each possible pair of symbols of $\Sigma' (= \Sigma \cup \{'-\'})$ is assigned a numerical value corresponding to the score for aligning the corresponding symbols. This representation provides an easy way to assign different scores to the alignments of different symbols. This is useful especially when dealing with protein alignments (but also with nucleotide), in which the

different pairs of amino acids have different degrees of chemical similarity. For this reason it is important to choose a scoring function that reflects biological observations when aligning protein sequences.

For this purpose, a set of scoring matrices has been designed to model the fact that some pairs of symbols are more probable than others in related sequences, by encoding the log-odd scores of amino acid mutations. In fact, although different types of scoring matrices have been proposed, such as those based on the similarity in terms of chemical properties, the ones obtained from empirical observations have shown to perform better than the others. The two most widely used matrices adopted for the pairwise alignment of protein sequences are Percent Accepted Mutation (PAM) ([Dayhoff and Schwartz, 1978](#)) and BLOSUM ([Henikoff and Henikoff, 1992](#)) matrices.

The former, i.e., the PAM matrices, have been computed by observing point mutation rates of amino acids in closely related proteins, with known phylogenies. The scores in the PAM1 matrix have been obtained by aligning closely related protein sequences, and estimate the expected rate of substitution if 1% of amino acids undergo substitutions. Starting from the PAM1 matrix, the PAM2 matrix is created by assuming an additional 1% of changes of the amino acids and, mathematically, this is done by multiplying the PAM1 matrix by itself. In general, this process can be repeated n time in order to obtain the PAM n matrix. The most widely used PAM matrices are PAM30, PAM70, PAM120, and PAM250. It must be pointed out that the scores of a PAM matrix are the logarithms (usually base 2) of the rate, averaged over the forward and backward substitution, and normalized to the rate of seeing a given amino acid (*log-odds scores*).

The other set of scoring matrices commonly used in the pairwise alignment of biological sequences are the BLOSUM matrices ([Henikoff and Henikoff, 1992](#)), which are based on the blocks database. These matrices consist of the logarithms (usually base 2) of the relative substitution frequencies of the aligned blocks from different protein families, representing a variety of organisms (*log-odds scores*). As for the PAM matrices, different BLOSUM matrices have been computed, each one having a different threshold of minimum similarity among the sequences of the organisms. More precisely, the BLOSUM N matrix is built by considering sequences having more than the $N\%$ of similarity as corresponding to a single sequence, thereby downweighting the information from highly similar sequences. One of the most used matrix is the BLOSUM62 matrix, which is built by clustering together and downweighting sequences with more than the 62% of identity ([Styczynski et al., 2008](#)).

Conclusions

In bioinformatics, one of the most classical problems is the alignment of two biological sequences, which is usually performed in order to assess similarities of DNA, RNA, or protein strings. Due to its importance, this problem has been thoroughly studied and many different variants proposed in literature. The two most important variants are global and local pairwise alignment. To solve these type of problems, the two most famous approaches are those proposed by Needleman and Wunsch, and by Smith and Waterman to compute optimal global and local pairwise alignment, respectively. Both techniques are based on dynamic programming algorithms, which find the optimal score of the pairwise alignment, with respect to a given scoring function, and also reconstruct an optimal alignment of the two input strings. In discussing the two recurrences on which the two algorithms are based, we provided also some an example for showing their functioning.

In this work, in addition to the linear gap model adopted in both the dynamic programming algorithms for solving the global and local pairwise alignments, we also discussed the affine gap model. Finally, we discussed some variants of the scoring function that are usually adopted in real case studies, in order to deal with biological sequences.

Further Reading

For a more detailed discussion on the pairwise alignment problem, we refer the reader to ([Jones and Pevzner, 2004](#)), to ([Gusfield, 1997](#)), and to ([Setubal and Meidanis, 1997](#)). These books provide a description of different methods for solving global and local pairwise alignment problems, in addition to the one presented in this work, and also give an extensive explanation of different scoring functions adopted for the alignment of biological sequences. In these books, heuristic methods for solving the aforementioned problems are discussed in detail. Moreover, a detailed analysis of gap penalties and different scoring functions is present in the aforementioned books.

References

- America, N., 2009. Inc: Metagenomics versus moores law. *Nature Methods* 6, 623.
- Bellman, R., 1954. The theory of dynamic programming. Technical Report. RAND CORP SANTA MONICA CA.
- Dayhoff, M.O., Schwartz, R.M., 1978. Chapter 22: A model of evolutionary change in proteins. In: Dayhoff, M.O. (Ed.), *Atlas of Protein Sequence and Structure*. Washington, DC: National Biomedical Research Foundation, pp. 345–352.
- Gotoh, O., 1982. An improved algorithm for matching biological sequences. *Journal of Molecular Biology* 162, 705–708.
- Gusfield, D., 1997. *Algorithms on Strings, Trees and Sequences: Computer Science and Computational Biology*. Cambridge University Press.
- Henikoff, S., Henikoff, J.G., 1992. Amino acid substitution matrices from protein blocks. *Proceedings of the National Academy of Sciences* 89, 10915–10919.
- Jones, N.C., Pevzner, P., 2004. *An Introduction to Bioinformatics Algorithms*. MIT Press.

- Medvedev, P., Stanciu, M., Brudno, M., 2009. Computational methods for discovering structural variation with next-generation sequencing. *Nature Methods* 6, S13–S20.
- Needleman, S.B., Wunsch, C.D., 1970. A general method applicable to the search for similarities in the amino acid sequence of two proteins. *Journal of Molecular Biology* 48, 443–453.
- Rodrigue, S., Materna, A.C., Timberlake, S.C., *et al.*, 2010. Unlocking short read sequencing for metagenomics. *PLOS ONE* 5, e11840.
- Schuster, S.C., 2008. Next-generation sequencing transforms today's biology. *Nature Methods* 5, 16.
- Sellers, P.H., 1974. On the theory and computation of evolutionary distances. *SIAM Journal on Applied Mathematics* 26, 787–793.
- Setubal, J.C., Meidanis, J., 1997. *Introduction to Computational Molecular Biology*. PWS Publishing.
- Smith, T.F., Waterman, M.S., 1981. Identification of common molecular subsequences. *Journal of Molecular Biology* 147, 195–197.
- Styczynski, M.P., Jensen, K.L., Rigoutsos, I., Stephanopoulos, G., 2008. Blo-sum62 miscalculations improve search performance. *Nature biotechnology* 26, 274–275.
- Tattini, L., DAurizio, R., Magi, A., 2015. Detection of genomic structural variants from next-generation sequencing data. *Frontiers in Bioengineering and Biotechnology* 3, 92.